Sprint 4 Documentation – OCR worker, MinIO

Team: Sebastian and Felix

Repository: https://github.com/BunnySupreme/SWEN3Project

Sprint duration: 20.11.2025 – 27.11.2025

Estimated workload: 12 h / person / week

1. Sprint Goal (from assignment)

1. Create the dedicated OCR Worker application running as a separate service.
2. Integrate an OCR engine (Tesseract/Ghostscript or equivalent) and demonstrate it with unit tests.
3. Extend the REST server to store uploaded PDF documents in MinIO (S3-compatible storage).
4. Implement the OCR Worker to:
    a. consume OCR job messages from RabbitMQ,
    b. fetch the original PDF from MinIO,
    c. perform OCR and generate a summary,
    d. publish the OCR result back to RabbitMQ / REST server.
5. Extend `docker-compose.yml` to run MinIO and the OCR Worker in containers.

2. Implemented Work

OCR Worker refactoring

1. Split the worker into:
    - OcrWorkerService (RabbitMQ connectivity and hosting).
    - OcrJobHandler (OCR workflow).
2. Introduced abstractions:
    - IFileStoreClient / MinioFileStoreClient for MinIO access.
    - IOcrEngine / TesseractOcrEngine for OCR execution.
3. Worker now:
    - Consumes OCR jobs from paperless.ocr.input.
    - Downloads the PDF from MinIO using the S3 API.
    - Runs Tesseract OCR to extract text.
    - Publishes an OCR result message to paperless.ocr.results.

SOLID / Clean Code

- Single Responsibility:
    - OcrWorkerService handles only message transport and lifecycle.

    - OcrJobHandler encapsulates business logic (download + OCR + result building).
    - MinioFileStoreClient and TesseractOcrEngine each have one clear responsibility.
- Dependency Inversion:
    - High-level workflow depends on interfaces (IOcrJobHandler, IFileStoreClient, IOcrEngine), which are injected via DI.
    - Concrete implementations (MinIO, Tesseract) can be replaced or mocked for unit tests.
- Testability:
    - OCR workflow can be unit-tested by mocking `IFileStoreClient` and `IOcrEngine` without RabbitMQ or real Tesseract.

MinIO Integration

1. Added paperless-minio service to docker-compose.yml with persistent volume.
2. Exposed S3 API on port 9000 and console on port 9001.
3. Added environment variables for MinIO endpoint, credentials, and bucket name.
4. Adjusted REST API upload path to store file bytes in MinIO.
5. Integrated fetching from MinIO into OCR Worker to allow summary generation
6. Integrated fetching from MinIO into REST and Frontend to allow download of stored files

REST Server

1. On document upload:
    a. Saves metadata to PostgreSQL.
    b. Stores the PDF in MinIO.
    c. Publishes an OCR job to RabbitMQ with `DocumentId` and object key.
2. RabbitConsumerService consumes OCR result messages and updates DocumentModel.Summary.
3. Added endpoint for file download

Quasar Frontend

1. Added download button and relevant methods for sending request to REST

Testing

1. Added unit tests for:

- OCR Worker's processing logic (given a message, calls MinIO and OCR engine, publishes result).
- MinIO file-storage integration (mocking S3 client).

6. Workload (Estimate)

Team Member	Week / Sprint	Estimated Hours	Main Tasks
Sebastian	Sprint 4	10	docker-compose, Frontend, MinIO, REST
Felix	Sprint 4	10	OCR Worker, Tesseract & Ghostscript